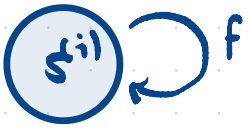
Clase 1: Modelado de Secuencias con RNN.

Las redes neuronales recurrentes (RNN siglas en inglés) fueron introducidas en 1986 por Rumelhart, Hilton y Williams en: "Learning representations by back-propagating errors", estas son una familia de redes neuronales dedicadas a procesar datos secuenciales (relaciones entre los datos) $x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(t)}$

Imaginemos que tenemos una serie de tiempo que proviene de un sistema dinámico

(1) $s^{(n)} = f^{(n)}(s^{(1)}; \theta)$

- $s^{(n)}$ estado n
- $f^{(n)}$ iteración del proceso
- $s^{(1)}$ estado inicial
- θ parámetros del sistema



¿Cómo podemos reescribir este sistema con ciclos (recurrencia) a un gráfo acíclico pero con una estructura repetitiva?

La ecuación (1) se puede reescribir como: $s^{(n)} = f(s^{(n-1)}; \theta)$ donde el estado actual depende del anterior; con esto en mente obtenemos ciertas ventajas:

- El modelo tiene el mismo tamaño de entrada sin importar el tamaño de la secuencia
- Utilizamos la misma función de transición con los mismos parámetros, lo que permite generalizar.

Arquitectura de las RNN

Consideremos $h^{(t)}$ un "vector", conocido como **estado oculto**, que funciona como un resumen de pérdida (lossy summary) de los aspectos relevantes de la secuencia hasta el estado (t).

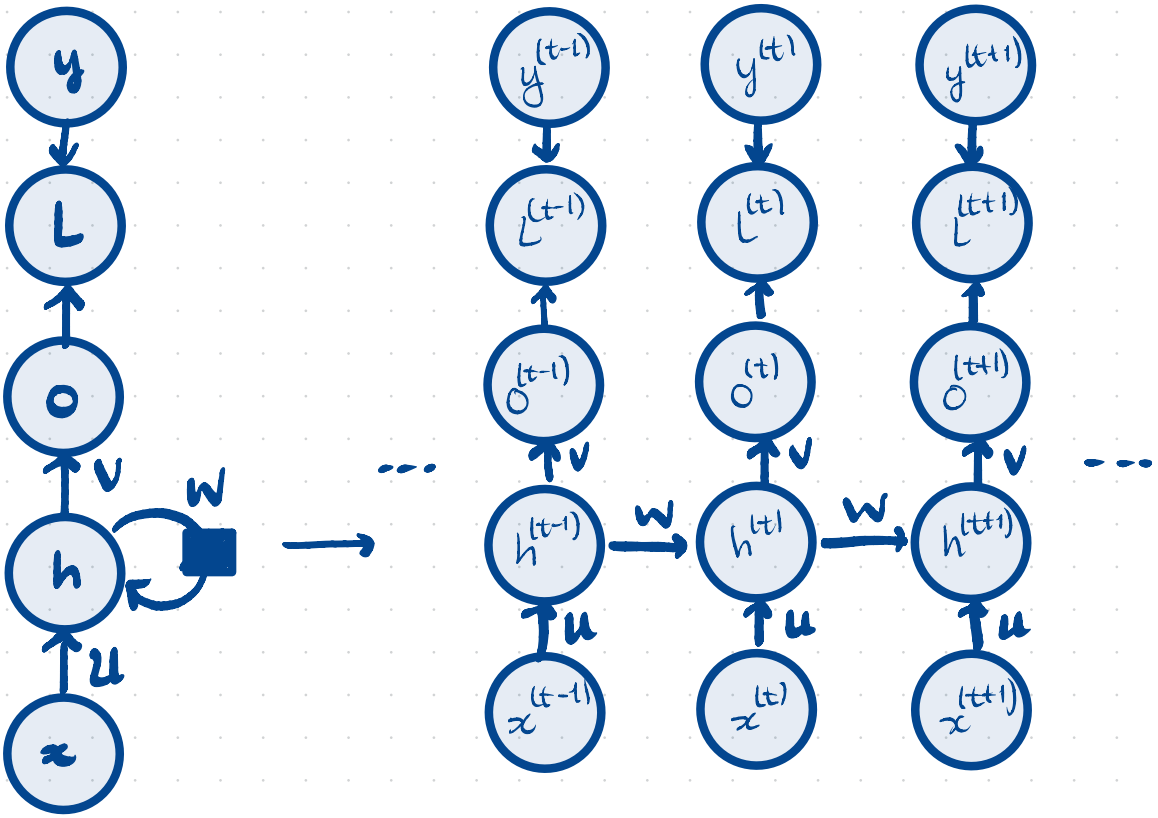
Activación: $a^{(t)} = b + W h^{(t-1)} + U x^{(t)}$

Actualización de Estado: $h^{(t)} = \Phi(a^{(t)})$

Salida: $o^{(t)} = c + V h^{(t)}$ $\quad \hat{y}^{(t)} = \psi(o_t)$

Matrices de Pesos

- U entrada a oculto
- V oculto a oculto
- W oculto a salida



¿Cómo garantizamos la "eficiencia" del modelo?

El loss total se construye a través de las perdidas en cada paso.

$$L = \sum_{t=1}^{\tau} \ell(y_t, \hat{y}_t) \qquad \ell(y, \hat{y}) = \begin{cases} \|y - \hat{y}\|^2 \\ -\sum_k y_k \log(\hat{y}_k) \end{cases}$$

↳ o_t para algunos casos

¿Cómo optimizamos?

Utilizaremos Retropropagación a través del tiempo (BPTT)

$$\nabla_{o^{(t)}} L = \frac{\partial \ell(y^{(t)}, \hat{y}^{(t)})}{\partial o^{(t)}}$$

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{\tau} \nabla_{o^{(t)}} L \cdot h^{(t)t} \qquad \frac{\partial L}{\partial c} = \sum_{t=1}^{\tau} \nabla_{o^{(t)}} L$$

$$\frac{\partial L}{\partial a^{(t)}} = \frac{\partial L}{\partial h^{(t)}} \odot \phi'(a^{(t)})$$

$$\frac{\partial L}{\partial h^{(t)}} = W^t \nabla_{o^{(t)}} L + V^t \cdot \frac{\partial L}{\partial a^{(t+1)}}$$

$$\rightarrow \frac{\partial L}{\partial a^{(t)}} = \left(W^t \nabla_{o^{(t)}} L + V^t \frac{\partial L}{\partial a^{(t+1)}}\right) \odot \phi'(a^{(t)})$$

$$\frac{\partial L}{\partial U} = \sum_{t=1}^{\tau} \frac{\partial L}{\partial a^{(t)}} \cdot x^{(t)t} \qquad \frac{\partial L}{\partial V} = \sum_{t=1}^{\tau} \frac{\partial L}{\partial a^{(t)}} \cdot h^{(t-1)t} \qquad \frac{\partial L}{\partial b} = \sum_{t=1}^{\tau} \frac{\partial L}{\partial a^{(t)}}$$

Algoritmo:

1. Calcular $a^{(t)}, h^{(t)}, o^{(t)}, \hat{y}^{(t)}$ para $t = 1, \ldots, \tau$.
2. Calcular $\frac{\partial \ell}{\partial o^{(t)}}$ para cada t.
3. Inicializar $\frac{\partial L}{\partial a^{(\tau)}} = 0$
4. Para $t = T$ hasta 1:
   - 4.1. $\left(\frac{\partial L}{\partial a^{(t)}} = W^t \frac{\partial \ell}{\partial o^{(t)}} + V^t \frac{\partial L}{\partial a^{(t+1)}}\right) \odot \phi'(a^{(t)})$
   - 4.2 Acumular $\frac{\partial L}{\partial W} += \frac{\partial \ell}{\partial o^{(t)}} \cdot h^{(t)t}$, $\frac{\partial L}{\partial U} += \frac{\partial L}{\partial a^{(t)}} \cdot x^{(t)t}$

     $\frac{\partial L}{\partial V} += \frac{\partial L}{\partial a^{(t)}} \cdot h^{(t-1)t}$, $\frac{\partial L}{\partial b} += \frac{\partial \ell}{\partial o^{(t)}}$, $\frac{\partial L}{\partial c} += \frac{\partial L}{\partial a^{(t)}}$

Patrones de Diseño

- Many-to-Many (mismo tamaño): Produce una salida en cada paso de tiempo con conexiones entre unidades ocultas.
- Many-to-One: Lee toda la secuencia y produce una única salida al final
- Seq2seq: Diseñadas para mapear secuencias de entrada a secuencias de salida de longitudes distintas.
  - ▸ Codificador: Procesa la entrada y emite un vector de contexto
  - ▸ Decodificador: Se condiciona en el vector de contexto para generar la secuencia de salida completa

Desafío de las dependencias a largo plazo: Gradiente Desvaneciente

¿Esto funciona siempre? No, al propagar el gradiente por muchas etapas estos tienden a desvanecerse o explotar.

¿Por qué pasa esto? Este problema se asemeja a multiplicar por una matriz de manera repetida. La respuesta dependerá fuertemente de los eigenvalores de esta.

¿Cuál es el impacto? La probabilidad de éxito cae rápidamente a casi cero para secuencias de apenas 10 o 20 pasos de longitud.

Documentación Pytorch:

Librería de tensores optimizada para el aprendizaje profundo.

- **torch.nn**: contiene los módulos para construir capas, incluyendo nn.RNN
- **torch.optim**: provee algoritmos de optimización como SGD y Adam.
- **torch.utils.data**: facilita el manejo de datos mediante TensorsDatasets y DataLoader, esenciales para procesar secuencias en mini-batches.

Implementación:

- Construcción manual: puedes usar nn.Linear para definir la celda y un bucle for para iterar sobre la secuencia, combinando el estado oculto y la entrada mediante torch.cat

- Uso de nn.RNN: es el módulo vainilla. Configura batch_first = True si los datos tienen la forma (batch, seq, feature)

Configuración de Tareas:

- Many-to-One (clasificación) se procesa toda la secuencia y solo se toma el último estado oculto (out[:,-1,:]) para pasar a una capa final (nn.Linear) que genera la predicción.

- Many-to-Many (etiquetado) se utilizan las salidas de todos los pasos de tiempo de la RNN para realizar una predicción por cada elemento de la secuencia.

Manejo de Gradientes y estabilidad.

- Funciones de Activación: se sugiere nn.Tanh pero podemos experimentar con nn.ReLU
- Funciones de Perdida: para una clasificación binaria es recomendable nn.BCEWithLogitsLoss.